

Fase 1 - Logika Penyelesaian Masalah

Pseudocode dan Flowchart

Nama: Zahra Yura Mahendra

Kelompok: Amethyst

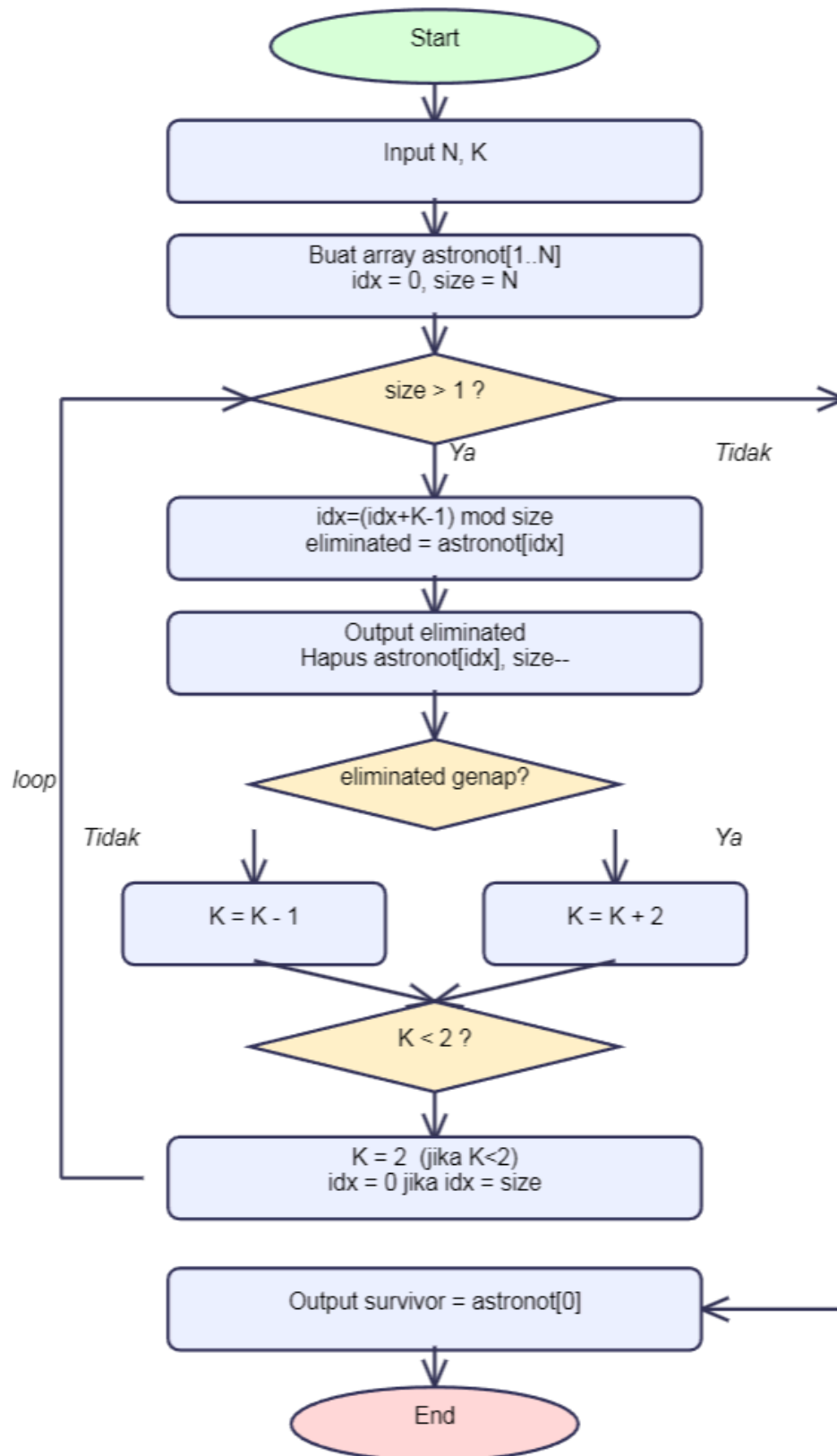
Problem 1: The Last Astronaut

Pseudocode:

```
START
INPUT N, K
CREATE array astronot [0..N-1] = {1,2,...,N}
size <- N
idx <- 0
WHILE size > 1 DO
    idx <- (idx+K-1) MOD size
    eliminated <- astronot[idx]
    OUTPUT eliminated
    REMOVE astronot[idx]
    size <- size - 1
    IF eliminated MOD 2 == 0 THEN
        K <- K + 2
    ELSE
        K <- K - 1
    ENDIF
    IF K < 2 THEN
        K <- 2
    ENDIF
    IF idx == size THEN
        idx <- 0
    ENDIF
ENDWHILE
OUTPUT "Survivor", astronot[0]

END
```

Flowchart:



Penjelasan Logika:

Persoalan ini merupakan variasi dari Josephus Problem. N astronot disimpan dalam sebuah array melingkar bernomor 1 sampai N. Sebuah variabel idx menandai posisi awal hitungan pada lingkaran, yang mula-mula menunjuk ke astronot nomor 1. Pada setiap putaran, posisi astronot yang mendapat hitungan ke-K dihitung dengan rumus $idx = (idx + K - 1) \bmod \text{size}$, di mana size adalah jumlah astronot yang masih tersisa di lingkaran. Astronot pada posisi tersebut dieliminasi dan dicatat urutannya, kemudian elemen-elemen setelahnya digeser ke kiri satu posisi sehingga idx secara otomatis menunjuk ke astronot yang berada tepat setelah posisi yang baru saja dieliminasi sesuai dengan aturan bahwa hitungan berikutnya dimulai dari sana.

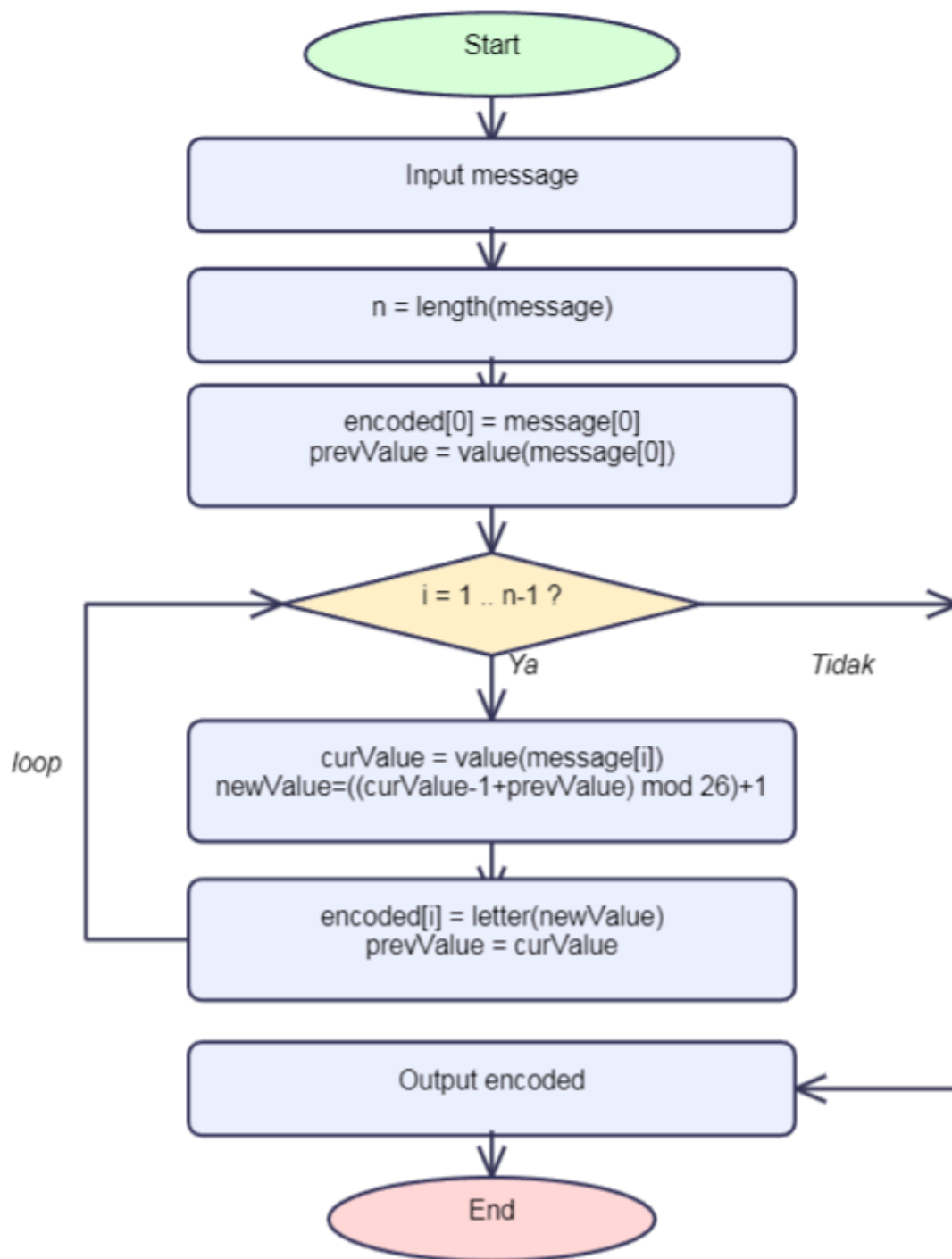
Setelah setiap eliminasi, nilai K diperbarui secara dinamis: jika nomor astronot yang dieliminasi genap maka K bertambah 2, sedangkan jika ganjil maka K berkurang 1. Untuk menjaga agar sistem eliminasi tetap valid, nilai K tidak diperbolehkan kurang dari 2 sehingga jika hasil perubahan membuat $K < 2$, nilai K langsung dipaksa menjadi 2. Proses perulangan ini terus dilakukan hingga hanya tersisa satu astronot di dalam array, yang kemudian ditetapkan sebagai astronot terakhir yang bertahan (survivor). Pendekatan simulasi langsung ini dipilih karena aturan perubahan K bergantung pada hasil eliminasi sebelumnya, sehingga tidak dapat diselesaikan dengan rumus Josephus klasik yang mengasumsikan K tetap.

Problem 2: Alien-In-The-Middle

Pseudocode:

```
START
INPUT message
n <- LENGTH(message)
encoded[0] <- message[0]
prevValue <- VALUE(message[0])
FOR i <- 1 TO n-1 DO
    curValue <- VALUE(message[i])
    newValue <- ((curValue - 1 + prevValue) MOD 26) + 1
    encoded[i] <- LETTER(newValue)
    prevValue <- curValue
ENDFOR
OUTPUT encoded
END
```

Flowchart:



Penjelasan Logika:

Sistem sandi ini mengubah setiap huruf pesan berdasarkan nilai posisi alfabet dari huruf sebelumnya yang asli (bukan huruf hasil sandi). Huruf pertama pesan tidak mengalami perubahan apa pun karena tidak memiliki huruf sebelumnya. Untuk huruf kedua dan

seterusnya, nilai posisi huruf saat ini ($A=1, B=2, \dots, Z=26$) dijumlahkan dengan nilai posisi huruf asli sebelum huruf tersebut, lalu hasilnya digunakan untuk menentukan huruf baru. Variabel `prevValue` selalu diperbarui menggunakan nilai huruf asli dari pesan input, bukan huruf yang sudah disandikan, sesuai dengan contoh pada soal di mana kolom 'Huruf Sebelumnya' selalu berisi huruf-huruf pesan asli (A, L, I, E, N, ...).

Karena alfabet hanya terdiri dari 26 huruf, penjumlahan nilai posisi dapat menghasilkan angka lebih dari 26 sehingga perlu dilakukan operasi modulo agar hasilnya tetap berada pada rentang 1 sampai 26 (wrap-around dari Z kembali ke A). Rumus yang digunakan adalah $\text{newValue} = ((\text{curValue} - 1 + \text{prevValue}) \bmod 26) + 1$; pengurangan dan penambahan 1 diperlukan karena operasi modulo standar bekerja pada rentang 0-25, sedangkan nilai huruf pada soal dimulai dari 1. Sebagai verifikasi, penerapan rumus ini pada contoh kata 'ALIENS' menghasilkan 'AMUNSG', sama persis dengan contoh yang diberikan pada soal.